

Projeto

quarta-feira, 26 de agosto de 2026 17:07

Site/Ferramentas para uso:

FLUXOGRAMA/ORGANOGRAMA:

- <https://excalidraw.com/>
- <https://app.diagrams.net/>
- https://whiteboard.cloud.microsoft/?source=applauncher&auth_upn=douglas.zanella%40aluno.senac.rio

Dados do Cliente:

Localização: Av. Lúcio Costa –Barra da Tijuca, RJ Espaço: 100 m² |

Ambientes climatizados e ao ar livre

Equipe: 70 funcionários (recepcionistas, garçons, cozinheiros e equipe de apoio)

Desafio: Automatizar os serviços com um sistema que gerencie:

- Cardápio (pratos, descrições e preços)
- Mesas e pedidos
- Garçons responsáveis pelos atendimentos

Objetivo do Sistema:

- Armazene pedidos, mesas, garçons e cardápio
- Otimize o processo de pedidos e pagamentos

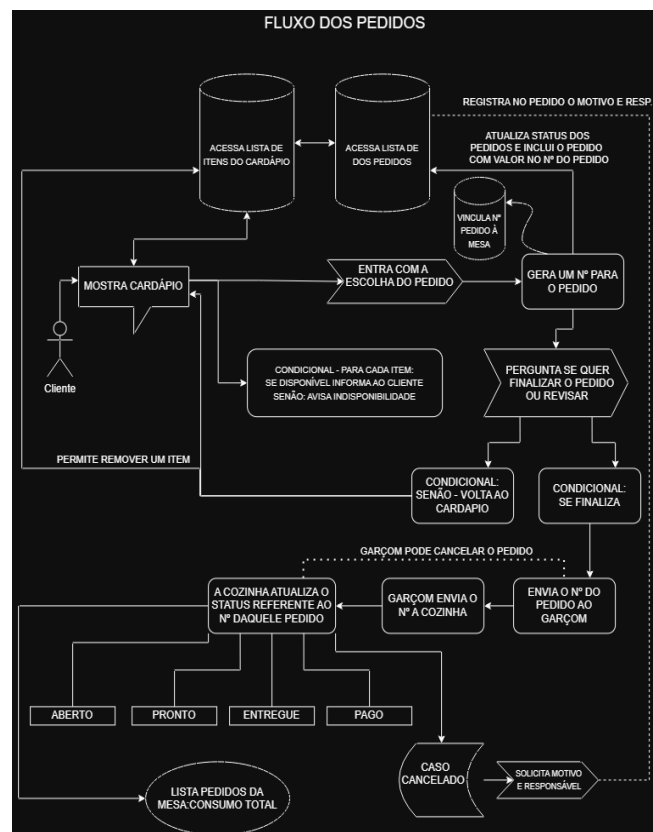
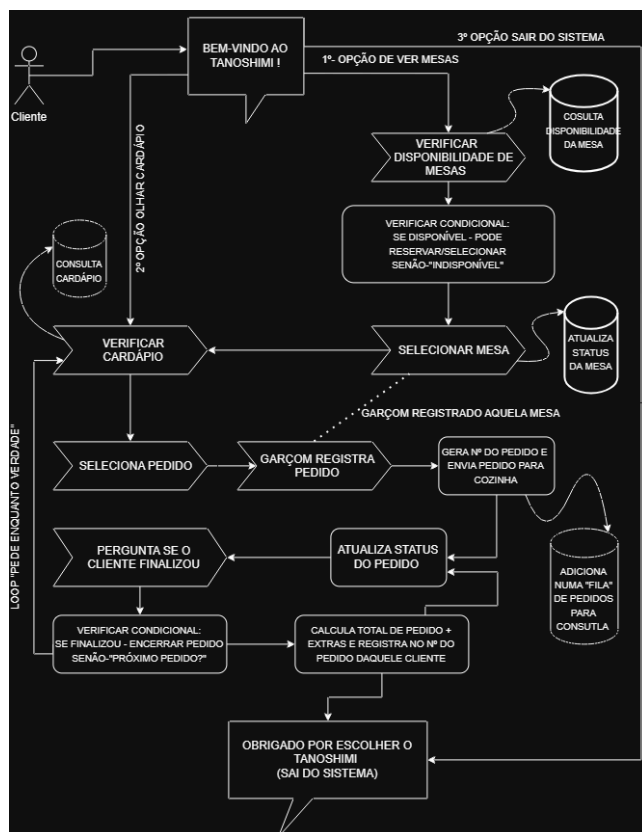
Fluxo de Atendimento:

- 1 Cliente acessa o cardápio digital (tablet ou celular)
- 2 Garçon realiza o pedido pelo sistema
- 3 Sistema gera um número de pedido (cartão entregue ao cliente)
- 4 Cliente apresenta o número na caixa para efetuar o pagamento

Minha Atribuição:

Consultar_Pedido()

Busca o pedido pelo Número do Pedido



Objetivo da função:

Consulta um pedido a partir do seu Nº, ou seja:

Existe um pedido com este número?

Se existe, quais são as informações atuais referentes a este pedido?

Raciocínio:

Os pedidos possivelmente serão armazenados em uma lista

(Lista_pedidos=[], por exemplo) e as informações relacionadas ao

Blocagem:

Ao receber a solicitação de "Consultar Pedido":

Verifica se a lista está vazia

Condicional:

Se sim: "Ainda não há pedidos!"

Senão: Continuar:

Receber ou solicitar o número do pedido

Ler numeração

Procurar esse número entre os pedidos registrados

pedidos serão dicionários.

Ou seja, devo criar uma função para percorrer a lista a procura do nº igual ao que foi solicitado

FUNÇÃO consultar_pedidos(numero_pedido, lista_pedidos)
 PARA cada pedido da lista_pedidos:
 SE o número do pedido for igual ao número procurado:
 RETORNAR o pedido encontrado
 SENÃO:
 RETORNAR pedido não encontrado(vazio)

```
#função consultar_pedido_python_ver1.0()
'''
para funcionar vou precisar importar funções externas, por exemplo:
from adicionar_pedido import adicionar_pedido
from remover_pedido import remover_pedido
from cancelar_pedido import cancelar_pedido
'''
#criação de uma lista "falsa" para testes::::
lista_pedidos = [
    {
        "numero_pedido": 20,
        "mesa": 4,
        "status": "PREPARANDO",
        "pagamento": "PENDENTE",
        "garcom": "José",
        "valor_total": 105.98
    },
    {
        "numero_pedido": 21,
        "mesa": 7,
        "status": "PRONTO",
        "pagamento": "PENDENTE",
        "garcom": "Cleiton",
        "valor_total": 75.90
    }
]

# Funções para testes
def remover_item(pedido):
    print(
        f"\nUm item seria removido do "
        f"pedido nº {pedido['numero_pedido']}."
    )

def adicionar_item(pedido):
    print(
        f"\nUm item seria adicionado ao "
        f"pedido nº {pedido['numero_pedido']}."
    )

def cancelar_pedido(pedido):
    pedido["status"] = "CANCELADO"
    print(
        f"\nO pedido nº {pedido['numero_pedido']} "
        f"foi cancelado."
    )

#-----FIM DO BLOCO DE TESTES -----
# def mostra_tela(pedido):
#     print(
#         "\n=====
#         "\n| SEU PEDIDO |"
#         "\n=====
#     )
#     print(f"Número: {pedido['numero_pedido']}")
#     print(f"Mesa: {pedido['mesa']}")
#     print(f"Garçom: {pedido['garcom']}")
#     print(f>Status: {pedido['status']}")
#     print(f"Pagamento: {pedido['pagamento']}")
#     print(f>Total: R$ {pedido['valor_total']:.2f}")
#
# def consultar_pedidos(numero_digitado, lista_pedidos):
#     '''
#     Função que procura o pedido dentro da lista de pedido
#     (essa lista pode ter sido criada quando a função "criar_pedido()" é
# executada,
#     esta função recebe o nº digitado e a lista criada como parâmetros
#     se encontrar a lista vazia, retornará "nada", se não encontrar o
# valor,
#     retornará que o pedido não existe, se encontrar um pedido equivalente
# ao número registrado,
```

OneNote

Tratar exceções, se o cliente digitou errado por exemplo:

"Entrada inválida"

Caso esteja tudo certo:

Condicional:

Se sim: apresentar ou devolver os dados do pedido

Retornar:

Número do pedido

Mesa do pedido

Garçom que atendeu

Itens no pedido

Total (R\$)

Status

Pagamento

Se sofreu alteração, informar o que foi alterado.

Senão: informar que o pedido não foi encontrado

Fim da consulta

```

        retornará o pedido
    ...
    consulta = "s"
    try:
        while consulta == "s":
            numero_digitado = int(input("Digite o número do pedido que deseja
consultar: "))
            if len(lista_pedidos) == 0:
                return None
            for pedido in lista_pedidos:
                if pedido["numero_pedido"] == numero_digitado:
                    return pedido
                consulta = input("Desejo continuar a consulta? (s/n)\n")
            return None
    except ValueError:
        print("Entrada inválida. Digite somente números.")
        return None

# # TESTE PARA VER SE A FUNÇÃO ESTÁ RETORTANDO CORRETAMENTE:::::
# while True :
#     pedido = consultar_pedidos()
#     if pedido != None:
#         print("\n===== DADOS DO PEDIDO =====")
#         print(f"Número: {pedido['numero_pedido']}")
#         print(f"Mesa: {pedido['mesa']}")
#         print(f"Garçom: {pedido['garcom']}")
#         print(f>Status: {pedido['status']}")
#         print(f"Pagamento: {pedido['pagamento']}")
#         print(f"Total: R$ {pedido['valor_total']:.2f}")

# print("\n=====")
# print("\nTecle 1 para: Adicionar mais itens"
# "\nTecle 2 para: Remover Itens"
# "\nTecle 3 para: Continuar com o Pedido atual"
# "\nTecle 4 para: Cancelar o Pedido"
# )
# print("\n=====")
# resp = input("Com o desejo prosseguir? -> TECLE 1 | 2 | 3 | 4")
# try:
#     match resp:
#         case 1:
#             adicionar_pedido(resp)
#         case 2:
#             remover_pedido(resp)
#         case 3:
#             cancelar_pedido(resp)
#         case _:
#             print("Entrada inválida")
# except ValueError:
#     print("Entada inválida, digite novamente")
pedido_encontrado = None #aqui o pedido inicia vazio, só passará a ter valor
quando a função for executado no while
continuar_consulta = "s" #define o valor para o while iniciar o loop
while continuar_consulta == "s":
    try:
        numero_digitado = int(input("\nDigite o número do pedido que deseja
consultar: "))
        pedido_encontrado = consultar_pedidos(numero_digitado, lista_pedidos)
        if pedido_encontrado is None: #se não tiver nada na lista ou não
encontrar o nº de pedido, ele ainda vai estar como None
            print(
                f"\nO pedido nº {numero_digitado} não foi encontrado.")
        else:
            mostrar_pedido(pedido_encontrado) #se achar, ele puxa lá da variável
que armazenou a execução da função com o numero_digitado
            continuar_consulta = input("\nDeseja realizar outra consulta? (s/n):
\n").lower() #lower para colocar em minusculo
            while continuar_consulta != "s" and continuar_consulta != "n": #valida se
foi digitado s ou n
                print("\nResposta inválida.")
                continuar_consulta = input("Digite apenas s ou n: ").lower()
    except ValueError:
        print(
            "\nEntrada inválida. O número do pedido deve conter somente
números.")

# Verifica se existe um pedido para poder dar opções ao usuário
if pedido_encontrado != None:
    print("\n=====")
    print("Tecle 1 para: Adicionar mais itens")
    print("Tecle 2 para: Remover itens")
    print("Tecle 3 para: Continuar com o pedido atual")
    print("Tecle 4 para: Cancelar o pedido")
    print("Tecle 0 para: Sair")
    print("=====")
    opcao = input("\nComo deseja prosseguir? ") #salva a resposta em opcao
    match opcao: #aqui entra a ligação com as demais funções::
        case "1":
            adicionar_item(pedido_encontrado)

```

```

    case "2":
        remover_item(pedido_encontrado)
    case "3":
        print(f"\n0 pedido nº {pedido_encontrado['numero_pedido']} continuará
sem alterações." )
    case "4":
        cancelar_pedido(pedido_encontrado)
    case "0":
        print("\nSaindo da consulta")
    case _:
        print("\n0 opção inválida.")
else: #se não tem um pedido diferente de None significa que não tem pedidos com
essa numeração a variável Pedido_encontrado ainda possui valor "vazio"
    print("\nPedido Inexistente")

```

Melhorias:

Conserto do loop, remover de dentro da função e chamar "por fora"

E resolver o problemas de não encontrar uma variável dentro da função

```

#função consultar_pedido_python_ver1.1()
#foram corrigidos erros nessa versão
'''

para funcionar vou precisar importar funções externas, por exemplo:
from adicionar_pedido import adicionar_pedido
from remover_pedido import remover_pedido
from cancelar_pedido import cancelar_pedido
'''

#criação de uma lista "falsa" para testes:::::
#-----BLOCO DE TESTES-----
-----
lista_pedidos = [
    {
        "numero_pedido": 20,
        "mesa": 4,
        "status": "PREPARANDO",
        "pagamento": "PENDENTE",
        "garcom": "José",
        "valor_total": 105.98
    },
    {
        "numero_pedido": 21,
        "mesa": 7,
        "status": "PRONTO",
        "pagamento": "PENDENTE",
        "garcom": "Cleiton",
        "valor_total": 75.90
    },
    {
        "numero_pedido": 22,
        "mesa": 14,
        "status": "ABERTO",
        "pagamento": "PENDENTE",
        "garcom": "José",
        "valor_total": 123.50
    }
]

# Funções para testes
def remover_item(pedido):
    print(
        f"\nUm item seria removido do "
        f"pedido nº {pedido['numero_pedido']}."
    )

def adicionar_item(pedido):
    print(
        f"\nUm item seria adicionado ao "
        f"pedido nº {pedido['numero_pedido']}."
    )

def cancelar_pedido(pedido):
    pedido["status"] = "CANCELADO"
    print(
        f"\n0 pedido nº {pedido['numero_pedido']} "
        "foi cancelado."
    )

#-----FIM DO BLOCO DE TESTES -----
-----
#FUNÇÃO DE CONSULTAR O PEDIDO (SIMPLES)
def consultar_pedido(numero_digitado, lista_pedidos):
    '''
    Função que procura o pedido dentro da lista de pedido
    (essa lista pode ter sido criada quando a função "criar_pedido()" é
    executada,
    esta função recebe o nº digitado e a lista criada como parâmetros
    se encontrar a lista vazia, retornará "nada", se não encontrar o valor,
    retornará que o pedido não existe, se encontrar um pedido equivalente ao
    número registrado,
    retornará o pedido
    '''

```

```

...
for pedido in lista_pedidos: #percorre a Lista em busca do pedido
    if pedido["numero_pedido"] == numero_digitado: #numero_pedido é o numero
que foi associado lá na criação
        return pedido #retorna os dados equivalente a este pedido
    return None #aqui caso não encontre ele precis retornar uma "resposta"
negativa(no casco vazia)
#-----FIM DA FUNÇÃO DE CONSULTA -----
#FUNÇÃO QUE MOSTRA AS INFORMAÇÕES NA TELA
def mostrar_pedido(pedido):
    valor_formatado = f"{pedido['valor_total']:.2f}"
    print("\n=====")
    print(f"PEDIDO Nº {pedido['numero_pedido']}")
    print("=====")
    print(f"Mesa: {pedido['mesa']}")
    print(f"Garçom: {pedido['garcom']}")
    print(f>Status: {pedido['status']}")
    print(f"Pagamento: {pedido['pagamento']}")
    print(f"Valor total: R$ {valor_formatado}")
    print("=====")

pedido_encontrado = None #aqui o pedido inicia vazio, só passará a ter valor
quando a função for executado no while
continuar_consulta = "s" #define o valor para o while iniciar o loop
while continuar_consulta == "s":
    try:
        ...
        ENTENDENDO A LINHA DE BAIXO:
        pega a informação do usuário(input)
        salva em numero_digitado
        em seguida executa a função de consulta baseada no numero digitado
        armazena dentro da variavel pedido_encontrado
        ...

        numero_digitado = int(input("\nDigite o número do pedido que deseja
consultar: "))
        pedido_encontrado = consultar_pedido(numero_digitado,lista_pedidos)
        if pedido_encontrado is None: #se não tiver nada na lista ou não
encontrar o nº de pedido, ele ainda vai estar como None
            print(
                f"\nO pedido nº {numero_digitado} não foi encontrado.")
        else:
            mostrar_pedido(pedido_encontrado)#se achar, ele puxa lá da variável
que armazenou a execução da função com o numero_digitado
            ...

            ENTENDENDO A LINHA ABAIXO:
            essa linha lê a resposta do usuário para continuar consultando ou não
            ...

            continuar_consulta = input("\nDeseja realizar outra consulta? (s/n):
\n").lower()#lower para colocar em minusculo
            while continuar_consulta != "s" and continuar_consulta != "n": #valida se
foi digitado s ou n
                print("\nResposta inválida.")
                continuar_consulta = input("Digite apenas s ou n: ").lower()
            except ValueError:
                print(
                    "\nEntrada inválida. O número do pedido deve conter somente
números.")

# Verifica se existe um pedido para poder dar opções ao usuário
if pedido_encontrado != None:
    print("\n=====")
    print("Tecle 1 para: Adicionar mais itens")
    print("Tecle 2 para: Remover itens")
    print("Tecle 3 para: Continuar com o pedido atual")
    print("Tecle 4 para: Cancelar o pedido")
    print("Tecle 0 para: Sair")
    print("=====")
    opcao = input("\nComo deseja prosseguir? ") #salva a resposta em opcao
    match opcao: #aqui entra a ligação com as demais funções::
        case "1":
            adicionar_item(pedido_encontrado)
        case "2":
            remover_item(pedido_encontrado)
        case "3":
            print(f"\nO pedido nº {pedido_encontrado['numero_pedido']} continuará
sem alterações." )
        case "4":
            cancelar_pedido(pedido_encontrado)
        case "0":
            print("\nSaindo da consulta")
        case _:
            print("\nOpção inválida.")
    else: #se não tem um pedido diferente de None significa que não tem pedidos com
essa numeração a variavel Pedido_encontrado ainda possui valor "vazio"
        print("\nPedido Inexistente")

```

2º Melhoria:

Encapsular as função em uma única que irá "chamar" as outras duas funções:

A ideia será:

Arquivo main chama a função consultar_pedido() e dentro do meu arquivo python, terá a função principal consultar_pedido que irá executar as duas funções auxiliares: procurar_pedido e mostrar_pedido:

```
# função consultar_pedido_python_ver1.2()
# foram feitos melhorias nessa versão
'''
para funcionar vou precisar importar funções externas, por exemplo:
from adicionar_pedido import adicionar_pedido
from remover_pedido import remover_pedido
from cancelar_pedido import cancelar_pedido
'''

# criação de uma lista "falsa" para testes::::
# -----BLOCO DE TESTES-----
-----
lista_pedidos = [
    {
        "numero_pedido": 20,
        "mesa": 4,
        "status": "PREPARANDO",
        "pagamento": "PENDENTE",
        "garcom": "José",
        "valor_total": 105.98
    },
    {
        "numero_pedido": 21,
        "mesa": 7,
        "status": "PRONTO",
        "pagamento": "PENDENTE",
        "garcom": "Cleiton",
        "valor_total": 75.90
    },
    {
        "numero_pedido": 22,
        "mesa": 14,
        "status": "ABERTO",
        "pagamento": "PENDENTE",
        "garcom": "José",
        "valor_total": 123.50
    }
]

# Funções para testes
def remover_item(pedido):
    print(
        f"\nUm item seria removido do "
        f"pedido nº {pedido['numero_pedido']}."
    )

def adicionar_item(pedido):
    print(
        f"\nUm item seria adicionado ao "
        f"pedido nº {pedido['numero_pedido']}."
    )

def cancelar_pedido(pedido):
    pedido["status"] = "CANCELADO"
    print(
        f"\nO pedido nº {pedido['numero_pedido']} "
        f"foi cancelado."
    )

# -----FIM DO BLOCO DE TESTES -----
-----

def procurar_pedido(numero_digitado, lista_pedidos):
    '''
    FUNÇÃO AUXILIAR
    Função que procura o pedido dentro da lista de pedido
    (essa lista pode ter sido criada quando a função "criar_pedido()") é
    executada,
    esta função recebe o nº digitado e a lista criada como parâmetros
    se encontrar a lista vazia, retornará "nada", se não encontrar o valor,
    retornará que o pedido não existe, se encontrar um pedido equivalente ao
    número registrado,
    retornará o pedido
    '''
    for pedido in lista_pedidos: # percorre a lista em busca do pedido
        numero_pedido é o numero que foi associado lá na criação
        if pedido["numero_pedido"] == numero_digitado:# retorna os dados
            equivalente a este pedido
            return pedido

    return None # aqui caso não encontre ele precisa retornar uma "resposta"
    negativa (no caso vazia)

def mostrar_pedido(pedido):
    '''
    FUNÇÃO AUXILIAR
    Mostra os dados do pedido encontrado.
    '''
    valor_formatado = f"{pedido['valor_total']:.2f}"
```

```

print("\n=====")
print(f"PEDIDO Nº {pedido['numero_pedido']}")
print("=====")
print(f"Mesa: {pedido['mesa']}")
print(f"Garçom: {pedido['garcom']}")
print(f"Status: {pedido['status']}")
print(f"Pagamento: {pedido['pagamento']}")
print(f"\nValor total: R$ {valor_formatado}")
print("=====")

def consultar_pedido(lista_pedidos):
    """
    Função principal da consulta.
    executa as duas funções auxiliares:
    Solicita o número, procura o pedido e mostra seus dados.
    Retorna o pedido encontrado ou None.
    """
    if len(lista_pedidos) == 0: # verificar se tem algum pedido na lista
        print("\nNão existem pedidos cadastrados.")
        return None
    pedido_encontrado = None # aqui o pedido inicia vazio, só passará a ter valor
    quando a função for executada no while
    continuar_consulta = "s" # define o valor para o while iniciar o loop
    while continuar_consulta == "s":
        try:
            """
            ENTENDENDO A LINHA DE BAIXO:
            pega a informação do usuário(input)
            salva em numero_digitado
            em seguida executa a função de consulta baseada no numero digitado
            armazena dentro da variavel pedido_encontrado
            """
            numero_digitado = int(input("\nDigite o número do pedido que deseja
consultar: "))
            pedido_encontrado = procurar_pedido(numero_digitado,lista_pedidos)
            if pedido_encontrado is None:# se não tiver nada na lista ou não
encontrar o nº de pedido,ele ainda vai estar como None
                print(
                    f"\nO pedido nº {numero_digitado} não foi encontrado.")

            else:#se achar, ele puxa lá da variável que armazenou a execução da
função com o numero_digitado
                mostrar_pedido(pedido_encontrado)
                """
                ENTENDENDO A LINHA ABAIXO:
                essa linha lê a resposta do usuário para continuar
                consultando ou não
                """
                continuar_consulta = input("\nDeseja realizar outra consulta? (s/n):
\n").lower() # lower para colocar em minusculo
                # valida se foi digitado s ou n:
                while (continuar_consulta != "s" and continuar_consulta != "n"):
                    print("\nResposta inválida.")
                    continuar_consulta = input("Digite apenas s ou n: ").lower()
                except ValueError:
                    print("\nEntrada inválida. Digite somente números.")
            # Verifica se existe um pedido para poder dar opções ao usuário
            if pedido_encontrado != None:
                print("\n=====")
                print("Tecle 1 para: Adicionar mais itens")
                print("Tecle 2 para: Remover itens")
                print("Tecle 3 para: Continuar com o pedido atual")
                print("Tecle 4 para: Cancelar o pedido")
                print("Tecle 0 para: Sair")
                print("=====")
                opcao = input("\nComo deseja prosseguir? ") # salva a resposta em opcao
                match opcao: # aqui entra a ligação com as demais funções
                    case "1":
                        adicionar_item(pedido_encontrado)
                    case "2":
                        remover_item(pedido_encontrado)
                    case "3":
                        print(f"\nO pedido nº {pedido_encontrado['numero_pedido']}
continuará sem alterações.")
                    case "4":
                        cancelar_pedido(pedido_encontrado)
                    case "0":
                        print("\nSaindo da consulta")
                    case _:
                        print("\nOpção inválida.")
            else:
                print("\nPedido Inexistente") #se não tem um pedido diferente de None
significa que não tem pedidos com essa numeração variável pedido_encontrado ainda
possui valor "vazio"
                return pedido_encontrado

# CHAMAR A FUNÇÃO PARA TESTAR:::
consultar_pedido(lista_pedidos)

```

O Código agora pesquisa por mais chaves:valores nos dicionarios, conforme documento indica:
 observações - itens- quantidade etc e inclui tambem forma de pagamento na consulta
 alem disso, divide o que é mostrado agora: - visao do cliente e visao do garçom

```
# função consultar_pedido_python_ver1.3()
# foram feitos melhorias nessa versão

'''
#para funcionar vou precisar importar funções externas, por exemplo:
from adicionar_item_pedido import adicionar_item_pedido
from remover_pedido import remover_pedido
from cancelar_pedido import cancelar_pedido
'''

# criação de uma lista "falsa" para testes::::
# -----BLOCO DE TESTES-----
# -----
# pedidos = [
#     {
#         "numero_pedido": 1,
#         "numero_da_mesa": 2,
#         "itens": "Sushi",
#         "quantidade": 8,
#         "status": "ABERTO",
#         "garcom": "Carlos",
#         "observacoes": "Sem wasabi",
#         "tipo_pagamento": "Débito",
#     },
#     {
#         "numero_pedido": 2,
#         "numero_da_mesa": 5,
#         "itens": "Temaki Salmão",
#         "quantidade": 3,
#         "status": "PREPARANDO",
#         "garcom": "Fernanda",
#         "observacoes": "Adicionar molho tarê",
#         "tipo_pagamento": "Débito",
#     },
#     {
#         "numero_pedido": 3,
#         "numero_da_mesa": 8,
#         "itens": "Combo Sushi Premium",
#         "quantidade": 1,
#         "status": "PRONTO",
#         "garcom": "Roberto",
#         "observacoes": "",
#         "tipo_pagamento": "Crédito",
#     },
#     {
#         "numero_pedido": 4,
#         "numero_da_mesa": 11,
#         "itens": "Hot Roll",
#         "quantidade": 12,
#         "status": "ENTREGUE",
#         "garcom": "Juliana",
#         "observacoes": "Cliente pediu guardanapos extras",
#         "tipo_pagamento": "PIX",
#     },
#     {
#         "numero_pedido": 5,
#         "numero_da_mesa": 14,
#         "itens": "Yakisoba de Frango",
#         "quantidade": 2,
#         "status": "PAGO",
#         "garcom": "Marcos",
#         "observacoes": "dobro de shoyu",
#         "tipo_pagamento": "VR-Refeição",
#     }
# ]

# # Funções para testes

# def remover_item(pedido):
#     print(
#         f"\nUm item seria removido do "
#         f"pedido nº {pedido['numero_pedido']}."
#     )

# def adicionar_item_pedido(pedido):
#     print(
#         f"\nUm item seria adicionado ao "
#         f"pedido nº {pedido['numero_pedido']}."
#     )
```



```

if len(pedidos) == 0: # verificar se tem algum pedido na lista
    print("\nNão existem pedidos cadastrados.")
    return None

pedido_encontrado = None # aqui o pedido inicia vazio, só passará a ter valor
quando a função for executada no while

continuar_consulta = "s" # define o valor para o while iniciar o loop
while continuar_consulta == "s":
    try:
        ...
        ENTENDENDO A LINHA DE BAIXO:
        pega a informação do usuário(input)
        salva em numero_digitado
        em seguida executa a função de consulta baseada no numero digitado
        armazena dentro da variavel pedido_encontrado
        ...

        numero_digitado = int(input("\nDigite o número do pedido que deseja
consultar: "))
        pedido_encontrado = procurar_pedido(numero_digitado,pedidos)

        if pedido_encontrado is None:# se não tiver nada na lista ou não
encontrar o nº de pedido,ele ainda vai estar como None
            print(
                f"\nO pedido nº {numero_digitado} não foi encontrado.")

        else:#se achar, ele puxa lá da variável que armazenou a execução da
função com o numero_digitado
            mostrar_pedido(pedido_encontrado)

        ...
        ENTENDENDO A LINHA ABAIXO:
        essa linha lê a resposta do usuário para continuar
        consultando ou não
        ...

        continuar_consulta = input("\nDeseja realizar outra consulta? (s/n):
\n").lower() # lower para colocar em minuscuro

        # valida se foi digitado s ou n:
        while (continuar_consulta != "s" and continuar_consulta != "n"):
            print("\nResposta inválida.")
            continuar_consulta = input("Digite apenas s ou n: ").lower()
        except ValueError:
            print("\nEntrada inválida. Digite somente números.")

# Verifica se existe um pedido para poder dar opções ao usuário
if pedido_encontrado != None:
    print("\n=====")
    print("Tecle 1 para: Adicionar mais itens")
    print("Tecle 2 para: Remover itens")
    print("Tecle 3 para: Continuar com o pedido atual")
    print("Tecle 4 para: Cancelar o pedido")
    print("Tecle 0 para: Sair")
    print("=====")

    opcao = input("\nComo deseja prosseguir? ") # salva a resposta em opcao

    match opcao: # aqui entra a ligação com as demais funções
        case "1":
            adicionar_item_pedido(pedido_encontrado)
        case "2":
            remover_item(pedido_encontrado)
        case "3":
            print(f"\nO pedido nº {pedido_encontrado['numero_pedido']}
continuará sem alterações.")
        case "4":
            cancelar_pedido(pedido_encontrado)
        case "0":
            print("\nSaindo da consulta")
        case _:
            print("\nOpção inválida.")
    else:
        print("\nPedido Inexistente") #se não tem um pedido diferente de None
significa que não tem pedidos com essa numeração variável pedido_encontrado ainda
possui valor "vazio"
        return pedido_encontrado

...

```

Lembrando que cada função vai depender da syntaxe do outro golega de turma
deverá ser ajustado para o nome que ele definir.

```
# CHAMAR A FUNÇÃO PARA TESTAR:::  
consultar_pedido(pedidos)
```